



웹 로그 데이터 수집 및 탐색적 분석(EDA)

7주차 2교시 — CSIC 2010 데이터셋으로 공격 패턴 직접 확인하기

WEEK 07 : DATA COLLECTION & EDA

🎯 2교시 학습 목표



데이터셋 이해

CSIC 2010 HTTP 데이터셋의 구조와 라벨 방식을 이해한다.



CSV 정제

Kaggle CSV를 받아 컬럼명 정규화·1만 건 샘플링하여 수업용 DataFrame을 만든다.



EDA 수행

정상 vs 공격의 차이를 시각화하며 1교시에 배운 키워드를 실제 데이터에서 확인한다.

💡 **오늘의 핵심 메시지:** 1교시에서 배운 SQL Injection (`' OR '1'='1'`), XSS (`<script>`), Path Traversal (`../`) 패턴을 실제 데이터에서 직접 눈으로 확인합니다!

</> 2교시 실습 코드 안내

파일 위치: `BigDataAnalysis/7_week/` · 분석은 JupyterLab에서 `.ipynb` 를 더블클릭, 발표용은 Streamlit

실습	파일명	내용	실행 방법
사전	<code>download_csic2010.py</code>	Kaggle CSV → 컬럼 정규화 + 1만 건 샘플링	<code>python download_csic2010.py</code>
실습 3	<code>data_load_explore.ipynb</code>	데이터 로드, 라벨/메서드 분포, 실제 HTTP 요청 확인	JupyterLab 더블클릭
실습 4-A	<code>eda_visualization.py</code>	Streamlit EDA 대시보드 (분포/길이/키워드/요청뷰어) — 발표용	<code>streamlit run eda_visualization.py</code>
실습 4-A	<code>eda_visualization.ipynb</code>	같은 차트의 노트북 버전 — 분석용 (Streamlit 의존성 없음)	JupyterLab 더블클릭
실습 4-B	<code>eda_keyword_analysis.ipynb</code>	공격 키워드 / URL 인코딩 / 경로 분석	JupyterLab 더블클릭

 분석은 JupyterLab(`.ipynb`)으로 시행착오 → 결과 공유는 Streamlit으로 패키징

CSIC 2010 데이터셋 소개

제작	스페인 국가연구위원회(CSIC)
목적	웹 애플리케이션 공격 탐지 연구
대상 앱	스페인어 쇼핑몰 (tienda1 = "가게1")
사용 형태	Kaggle 배포 CSV — <code>csic_database.csv</code>
라벨	Normal / Anomalous
원본 크기	정상 36,000건 + 공격 25,065건 = 61,065건
수업 사용	1만 건 샘플 (라벨 비율 유지)

👁 왜 이 데이터셋인가?

```
# [CSIC 2010 (HTTP 요청 텍스트)]
GET /login?user=admin'-- HTTP/1.1
Host: localhost:8080
Cookie: JSESSIONID=ABC123
```

- 공격 패턴이 눈에 보임!
- 1교시 키워드 직접 확인

텍스트 기반이라 LLM 활용에도 최적 (10주차)

파일 구성 & 공격 유형

데이터셋 파일 구성

파일명	위치	내용
csic_database.csv	data/archive/	Kaggle 원본 (61,065건, 17컬럼)
csic2010_requests.csv	7_week/	수업용 정제본 — <code>download_csic2010.py</code> 실행으로 자동 생성 (1만 건)

포함된 공격 유형

SQL Injection

파라미터에 SQL 코드 삽입

XSS

`<script>` 등 자바스크립트 삽입

Path Traversal

`../` 로 허용 외 파일 접근

CRLF Injection

`%0d%0a` 로 헤더 조작

Buffer Overflow

비정상적으로 긴 문자열 입력

Parameter Tampering

가격·수량 파라미터 변조

⚠ 주의: 개별 요청에 공격 유형 라벨은 없음 — 정상/비정상만 구분. 어떤 공격인지는 우리가 분석해서 알아내야 함!

🔗 HTTP 요청은 원래 어떻게 생겼는가?

CSV로 가공되기 전 원시 HTTP 텍스트 형태. 각 컬럼이 어디서 왔는지 이해하면 분석이 쉬워집니다.

GET 요청 예시

```
GET /tienda1/publico/anadir.jsp?id=2&cantidad=1 HTTP/1.1 ← 요청 라인
Host: localhost:8080 ← 헤더 시작
User-Agent: Mozilla/5.0 (compatible; ...)
Accept: text/xml,application/xml,...
Cookie: JSESSIONID=E1A1E4B0A0D68EA62... ← 헤더 끝
                                     ← (빈 줄)
```

POST 요청 예시 (본문 포함)

```
POST /tienda1/publico/autenticar.jsp HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 64
                                     ← (빈 줄 = 헤더와 본문 구분)
usuario=admin&contrasena=1234&B1=Entrar ← POST 본문 (Body)
```

 Kaggle 버전은 위 텍스트가 이미 **17개 컬럼으로** 분리되어 있습니다. 우리는 컬럼명 표준화 + 1만 건 샘플링만 수행.

Kaggle CSV의 컬럼 구조

Kaggle 원본 컬럼	정제 후 컬럼	설명	예시
(이름 없음)	label	라벨 (Normal/Anomalous)	Normal
Method	method	HTTP 메서드	GET, POST, PUT
URL	url + http_version	URL에서 host와 HTTP/1.1 분리	/tienda1/...?id=2
host	host	호스트	localhost:8080
User-Agent	user_agent	클라이언트 정보	Mozilla/5.0...
cookie	cookie	쿠키 값	JSESSIONID=...
content-type	content_type	POST 본문 타입	application/x-www-form-urlencoded
lenght (원본 오타)	content_length	본문 길이	64
content	body	POST 본문	usuario=admin&...
classification	classification	숫자 라벨	0(정상) / 1(공격)

🔄 사전 준비 — Kaggle CSV → 수업용 CSV

1 Kaggle 데이터 다운로드 후 배치

📄 Kaggle 링크: [CSIC 2010 Web Application Attacks](#) — Download 버튼 클릭 (Kaggle 계정 필요)

```
BigDataAnalysis/7_week/  
└─ data/archive/csic_database.csv    ← 여기에 저장
```

2 정제 스크립트 실행

```
python download_csic2010.py          # 1만 건 샘플 (수업용 - 권장)  
python download_csic2010.py --full   # 전체 61,065건
```

🕒 왜 1만 건만? 6만 건은 Streamlit 응답이 느림. 1만 건이면 EDA가 즉각 반응 + 라벨 비율과 공격 다양성 유지.

🔄 재현성: `random_state=42` 고정 — 누가 실행해도 동일한 1만 건이 나옴.

</> 핵심 처리 코드 — download_csic2010.py

```
# 파일: download_csic2010.py
import pandas as pd
import re

# Kaggle CSV 로드
raw = pd.read_csv("data/archive/csic_database.csv")

# 컬럼 정규화 (라벨, 메서드, URL 등)
df = pd.DataFrame()
df["label"] = raw.iloc[:, 0]          # 첫 컬럼 = Normal/Anomalous
df["method"] = raw["Method"]
df["body"] = raw["content"]          # POST 본문
df["classification"] = raw["classification"] # 0/1

# URL 정제: 'http://localhost:8080/path?q HTTP/1.1' → '/path?q'
def clean_url(s):
    s = re.sub(r"^https?:\/\/[^\s]+", "", str(s))    # 호스트 제거
    s = re.sub(r"\s+HTTP/[^\s]+\s*$", "", s)        # 버전 제거
    return s.strip()

df["url"] = raw["URL"].apply(clean_url)

# 라벨 비율 유지하며 1만 건 샘플링
normal = df[df["label"] == "Normal"].sample(n=5895, random_state=42)
```

분석 환경 — JupyterLab vs Streamlit

데이터 분석은 "탐색"과 "공유" 두 단계 — 각각에 적합한 도구가 다르다

[탐색/분석] JupyterLab

- .ipynb 노트북 셀 단위 실행
- Shift+Enter로 셀별 진행
- 변수 상태 메모리 유지
- 시행착오 자유롭게
- 실습 3, 4-A(분석용), 4-B 적합

[발표/공유] Streamlit

- 인터랙티브 대시보드
- 위젯으로 비개발자도 사용
- 최종 결과물 패키징
- 발표/제출용
- 실습 4-A(발표용) 적합

↔ **실행 모델 차이:** JupyterLab은 셀 단위 → 변수 누적(Bottom-up). Streamlit은 위젯이 바뀔 때마다 스크립트 전체 재실행(Top-down).

天秤 상황별 도구 비교

상황	JupyterLab (셀 모드)	Streamlit
<code>df.head()</code> 한 번 보고 싶음	셀 실행 1초	앱 재시작/리로드 필요
새 컬럼 1개 추가 실험	셀에서 바로	코드 수정 → 저장 → 리로드
변수 상태 유지	✔ 메모리에 보관	✘ 매번 처음부터 재실행
중간 결과를 다음 셀에서 활용	✔ 자연스러움	✘ <code>session_state</code> 필요
<code>print()</code> 디버깅	✔ 즉시 출력	✘ <code>st.write()</code> 로 변환
모델 학습 (5분 소요)	✔ 1번만 실행	✘ 위젯 변경 시 재학습
비개발자에게 공유	✘ 코드 노출	✔ 깔끔한 웹 UI

💡 분석가는 "데이터에 무엇이 있는지" 모르는 상태에서 시작 — 가설↔코드↔결과 반복 루프가 핵심. JupyterLab이 이 루프 속도를 극대화.

↓ JupyterLab — 설치 + 실행

🛡️ ⚠️ 시작 전 — venv 가상환경부터! venv 가상환경을 활성화부터 시킨 후 실행해주세요.

1 JupyterLab 설치 (venv 활성화 상태에서)

```
pip install jupyterlab
```

2 분석 패키지 설치

```
pip install pandas numpy plotly streamlit
```

3 JupyterLab 실행

```
jupyter lab  
# → http://localhost:8888/lab 자동 열림
```

🔗 자주 쓰는 옵션

<code>--port=8889</code>	포트 변경
<code>--no-browser</code>	브라우저 자동 실행 안 함
<code>--notebook-dir=경로</code>	시작 디렉토리 지정
종료	터미널에서 <code>Ctrl+C</code> 두 번

📁 좌측 파일 트리에서 `BigDataAnalysis/7_week/` 로 이동 → `.ipynb` 더블클릭

📄 노트북 사용법

좌측 상단  버튼 → "Python 3" 커널 선택. 셀별 **Shift+Enter**로 실행.

```
# 셀 1 - 데이터 로드
import pandas as pd
df = pd.read_csv("csic2010_requests.csv")
df.head()           # 표 즉시 출력

# 셀 2 - 라벨 분포
df['label'].value_counts()

# 셀 3 - URL 길이 그룹별 통계
df['url_length'] = df['url'].str.len()
df.groupby('label')['url_length'].describe()
```

📄 단축키 요약

Shift+Enter	셀 실행 후 다음 셀로
Ctrl+Enter	현재 셀만 실행
Esc + A / B	위/아래에 새 셀 추가
Esc + M / Y	Markdown / Code 변환
Esc + D, D	셀 삭제

 출력은 셀 바로 아래에 누적 표시. 변수는 메모리에 유지되어 다음 셀에서 그대로 사용.

📍 수업 권장 흐름

1 실습 3 — `data_load_explore.ipynb`

JupyterLab에서 더블클릭 → 셀 10개를 위에서부터 Shift+Enter로 순차 실행 — 데이터 로드/라벨/메서드/실제 요청 확인

2 실습 4-B — `eda_keyword_analysis.ipynb`

셀 6개 단위 실행 — 키워드 / 인코딩 / 경로 분석

3 실습 4-A 분석용 — `eda_visualization.ipynb`

Plotly 차트(파이/막대/히스토그램/Box)를 셀 13개 단위로 만들며 학습

4 실습 4-A 발표용 — `eda_visualization.py`

별도 터미널에서 `streamlit run` 으로 같은 분석을 인터랙티브 대시보드로 정리한 모습 시연

→ 분석은 JupyterLab(1~3)으로 시행착오 → 공유는 Streamlit(4)으로 패키징

→ 데이터 로드 + 라벨 분포 ★

데이터 로드

```
# 파일: data_load_explore.ipynb
import pandas as pd
df = pd.read_csv('csic2010_requests.csv')
print(f"데이터 크기: {df.shape}")
print(f"전체 요청 수: {len(df):,}건")
```

라벨 분포 확인

```
print(df['label'].value_counts())

# 예상 출력 (1만 건 샘플 기준):
# Normal      5,895건 (약 59%)
# Anomalous   4,105건 (약 41%)
```

⚠ 관찰 + ★ 핵심

정상 59% : 공격 41% — 불균형 데이터

★ **정확도(Accuracy)의 함정**: 모든 요청을 정상 예측하면 정확도 59% — 그러나 공격을 단 한 건도 못 잡음! 불균형 데이터는 **F1-Score**나 **혼동행렬**을 함께 봐야 함.

★ **보안에서는 미탐(FN)이 더 위험**: "정상인을 도둑으로 오인"은 작은 불편이지만, "진짜 도둑을 놓침"은 DB 통째로 털리는 사고. → 보안 분야에선 **재현율(Recall)**을 중시.

🔍 HTTP 메서드 + 실제 요청 확인 ★★

HTTP 메서드 분포

```
# 파일: data_load_explore.ipynb
print(df['method'].value_counts())

# GET      6,984건
# POST     2,945건
# PUT       71건
```

실제 HTTP 요청 비교

```
# 정상 요청 1건 확인
normal = df[df['label'] == 'Normal'].iloc[0]
print("=== 정상 요청 ===")
print(f"메서드: {normal['method']}")
print(f"URL: {normal['url']}")

# 공격 요청 1건 확인
attack = df[df['label'] == 'Anomalous'].iloc[0]
print(f"메서드: {attack['method']}")
print(f"URL: {attack['url']}")
```

✅ 정상 요청

GET /tienda1/imagenes/3.gif HTTP/1.1

💀 공격 요청

GET /tienda1/...?id=2'+OR+'1'='1 HTTP/1.1

👁️ 1교시 키워드를 직접 찾아보세요! ' , OR , 1=1 , <script> , ../

🏰 4-1. URL 길이 분석

길이 통계 + 히스토그램

```
# 파일: data_load_explore.ipynb (셀 8)
df['url_length'] = df['url'].str.len()

print(f"정상 URL 평균: {df[df['label']=='Normal']['url_length'].mean():.0f}")
print(f"공격 URL 평균: {df[df['label']=='Anomalous']['url_length'].mean():.0f}")

# 파일: eda_visualization.ipynb (셀 3)
import plotly.express as px

fig = px.histogram(
    df, x='url_length', color='label',
    title='URL 길이 분포 (정상 vs 공격)',
    barmode='overlay', opacity=0.7, nbins=100,
    color_discrete_map={
        'Normal': '#22d3ee',
        'Anomalous': '#ef4444'
    }
)
fig.show()
```

🎯 **관찰:** 공격 요청의 URL이 평균적으로 더 길다. SQL Injection / XSS 코드가 URL에 삽입되면서 길이가 늘어남.

⚡ **10주차 미리보기 — IQR 이상치:** 정상은 좁은 범위에, 공격은 오른쪽 꼬리에 집중 → "정상 분포 범위를 벗어나면 이상치"라는 IQR 원리의 근거.

⚠️ **길이만으로 판단 금지:** 긴 검색어, 다국어 입력 등 긴 정상 요청도 존재. 길이는 단서일 뿐 — 다음 시간 특성 엔지니어링에서 키워드/특수문자/엔트로피와 조합.

4-2. POST 본문 길이 분석

```
# 파일: eda_visualization.ipynb (셀 5)
df['body_length'] = df['body'].fillna('').str.len()

# POST 요청만 필터링
post_df = df[df['method'] == 'POST']

fig = px.box(
    post_df,
    x='label', y='body_length',
    title='POST 본문 길이 (정상 vs 공격)',
    color='label',
    color_discrete_map={
        'Normal': '#22d3ee',
        'Anomalous': '#ef4444'
    }
)
fig.show()
```

☑ 왜 Box Plot인가?

- 중앙값(median) + 사분위수(Q1, Q3) 한눈에
- 이상치(점)가 자동으로 표시됨
- 정상/공격 분포 비교가 명확

 POST 본문은 로그인/검색/주문 등 입력값이 들어가는 부분 — 공격이 집중되는 곳!



4-3. 공격 키워드 빈도 ★★

```
# 파일: eda_keyword_analysis.ipynb (셀 3)
attack_keywords = {
    'SQL Injection': ['"', 'OR', 'UNION', 'SELECT', 'DROP', '--', '1=1'],
    'XSS': ['<script', 'alert(', 'onerror', '<iframe', '<img', 'javascript:'],
    'Path Traversal': ['../', '%2e%2e', '/etc/passwd', '/etc/shadow'],
    'Command Injection': [';', '|', '&&', '/bin/', 'cat ', 'rm ']
}

# 각 요청의 URL+본문에서 키워드 검색
full_text = df['url'].fillna('') + ' ' + df['body'].fillna('')

for category, keywords in attack_keywords.items():
    count = 0
    for kw in keywords:
        count += full_text.str.contains(kw, case=False, na=False).sum()
    print(f"{category}: {count}건에서 관련 키워드 발견")
```

⚙️ case=False를 쓰는 이유 — 보안의 역동성

- 공격자는 `SELECT` 를 `sELeCt` 처럼 대소문자 섞어 단순 필터를 우회 — SQL은 대소문자 무시이므로 그대로 작동
- `case=False` 로 이 단순 우회를 막을 수 있음
- 공격자는 또 인코딩 우회 (`%53ELECT`) · 주석 삽입 (`SEL/**/ECT`)으로 진화 → 방어자는 또 새 탐지 — 끝없는 공격-방어 대결

4-4-4-5. URL 인코딩 & 경로 분석

4-4. URL 인코딩 패턴

```
# 파일: eda_keyword_analysis.ipynb (셀 5)
encoded_patterns = ['%27', '%3C', '%3E',
                   '%2F', '%0d', '%0a', '%00']

for p in encoded_patterns:
    n = df[df['label']=='Normal']['url'].str.contains(p, na=False).sum()
    a = df[df['label']=='Anomalous']['url'].str.contains(p, na=False).sum()
    print(f"{p}: 정상 {n}건, 공격 {a}건")
```

`%27` = ' | `%3C` = < | `%2F` = / | 인코딩된 공격 패턴이 공격 요청에 집중됨

4-5. 요청 경로 분석

```
# 파일: eda_keyword_analysis.ipynb (셀 6)
df['path'] = df['url'].str.split('?').str[0]
df['path_depth'] = df['path'].str.count('/')

# 가장 많이 접근된 경로 Top 10
print(df['path'].value_counts().head(10))

# 공격에서 자주 접근된 경로
attack_paths = df[df['label']=='Anomalous']['path'].value_counts().head(10)
print(attack_paths)
```

💡 특정 경로(로그인, 검색, 인증 등 입력이 들어가는 페이지)에 공격이 집중된다는 것을 직접 확인

🚩 2교시 핵심 정리

단계	내용	포인트
데이터셋	CSIC 2010 = 웹 공격 탐지용 HTTP 요청 데이터	Kaggle CSV로 제공
CSV 정제	컬럼명 정규화 + 1만 건 샘플링	URL에서 host/HTTP 버전 분리
기본 탐색	건수, 메서드 분포, 라벨 비율	정상 59% : 공격 41%
EDA	URL 길이, 키워드 빈도, 경로 분석	공격 패턴이 텍스트에서 직접 보임!

📌 핵심 발견 4가지

- 발견 1** — 공격 요청의 URL이 평균적으로 더 김 → 길이 단독으로도 1차 분류 가능
- 발견 2** — 1교시 키워드(' , `SELECT` , `<script>` , `../`)가 실제 데이터에서 확인됨
- 발견 3** — URL 인코딩 패턴(`%27` , `%3C`)이 공격 요청에서 집중적으로 나타남
- 발견 4** — 특정 경로(로그인/검색)에 공격 집중 → 입력 받는 페이지가 표적

🏆 마무리 챌린지 + 다음 시간 예고

데이터 훑어보기 챌린지 (1분)

무작위 5개 요청을 보고 '공격이라고 확신하는 것'을 찾아 번호 + 근거 한 줄을 채팅창에 작성!

```
sample = df.sample(5, random_state=42)
for i, row in sample.iterrows():
    print(f"\n#{i} {row['method']} {row['url'][:120]}")
    if row['body']:
        print(f"      BODY: {row['body'][:100]}")
```

→ 학생이 직접 `'`, `<script>`, `../`, `UNION` 을 눈으로 찾아내며 1·2교시 학습이 실전에서 작동함을 체감

→ 3교시 예고: 텍스트 데이터에서 머신러닝이 학습할 수 있는 숫자형 특성(**Feature**)을 직접 추출. URL 길이, 키워드 존재 여부, 특수문자 개수 등 → DataFrame의 새 컬럼으로!

